

Specification for Implementation of Stack Canaries in S3K

1 What This Project Aims to Protect Against

Buffer overflow attacks utilize bugs commonly found in non-memory-safe code. Using functions that read or write data from memory without limiting access to set bounds enables reading or writing to a larger part of memory than intended. This can give an attacker power over the control flow of a process, as well as enable remote code execution by writing their own code to memory before it is executed. Because of its severity, several different ways to protect against this type of attack have been created, usually used together to increase the security of the operating system. The goal is to protect the operating system, as a process cannot be considered trusted space.

2 What Is a Stack Canary?

Stack canaries are one of many measures that can be used as buffer overflow protection. Canaries are spaces filled with a value known to a monitor. When the value changes during runtime, the monitor knows that behavior not inherent to the program has occurred and can act to protect the operating system, usually by terminating the process. There are different types of canaries used to protect the stack.

2.1 Terminator Canaries

Terminator canaries protect through the knowledge that most buffer overflow attacks use string operations that end at string terminators. The space allocated to the canary is filled with string terminators, such as null terminators, carriage returns, and page breaks. This will interrupt reading operations such as `strcpy()` when reading past the space allocated to the string on the stack.

2.2 Random Canaries

Random canaries fill the allocated space on the stack with a randomized value. The value is often gathered from entropy to make discovering it implausible. The canary is a secure value known only to the function that needs to know it: the stack canary monitor. To bypass this type of canary, the attacker must find a way to read from the stack and then place the correct canary back as the stack is overwritten.

2.3 Random XOR Canaries

Random XOR canaries are random canaries with additional security implemented to make overwriting the canary more difficult. These canaries use an XOR algorithm applied to a larger part of the control data together with the canary value when performing the comparison. This effectively forces the attacker to read a far larger part of memory before being able to interfere with the control flow.

3 Project Idea

The project will encompass implementing random canaries into the control flow of the S3K kernel.

3.1 Likely Steps Needed for the Canary

1. Generate entropy for a randomized number and create the number on process start.
2. Allocate and write the canary to each function on the stack before the stack pointer and return address.
3. Place the canary monitor inside a suitable handler that already interrupts the process flow, or create a new hook at certain actions where the canary monitor can check the value.
4. Implement error handling for when the monitor deems the function tampered with, including a label indicating why the function exited.

It would be reasonable to edit parts of the processor code that already hook at suitable moments and add the required code, instead of creating an unnecessarily complex workflow.

The canary function generates one canary string per process and then reuses it in every function to keep the overhead small. The string is created on process start. It would be reasonable to add the code to the process-start and declaration function that sets up the process and PCB.

Source

- https://en.wikipedia.org/wiki/Buffer_overflow_protection